

# Foundation-Model Fine-Tuning Results — Demand Forecasting

**Name:** Yahya Hammoudeh  
**Date:** 2026-04-14  
**Supervisor:** Mohammed Reda

---

## Executive Summary

We attempted to beat our tree-based baseline (XGBoost, 5.91M DKK cost) by fine-tuning Chronos-Bolt-Mini — Amazon’s 21M-parameter time-series foundation model. Result:

| Attempt                                | Best test cost (DKK) | Training time      | vs XGB cap=3 (6.56M) |
|----------------------------------------|----------------------|--------------------|----------------------|
| <b>Chronos pre-trained (zero-shot)</b> | <b>8,962,024</b>     | 0 (no training)    | <b>+36.6%</b>        |
| <b>Chronos fine-tuned (10 epochs)</b>  | <b>9,194,415</b>     | 32 minutes on A100 | <b>+40.2%</b>        |

**Fine-tuning actively hurt Chronos** (−2.6% from pre-trained). Scaling up (Tiny → Base, 9M → 205M params) did not change the outcome either.

**Verdict: foundation models are the wrong tool for this problem.**

---

## What We Tried

### 1. Chronos-Bolt-Mini fine-tune (main attempt)

**Setup:** - Base: amazon/chronos-bolt-mini, 21M params - Context length: 64 days, prediction horizon: 3 days - Outlier clipping at 99th percentile per series (73 units/day cap) - Implicit Huber-like loss (Chronos internal) - 10 epochs, batch size 64, AdamW lr=5e-5, cosine decay, early stopping patience=3 - A100 training: ~186s per epoch, 31 minutes total

**Training trajectory:**

| Epoch | Train loss | Val loss | Verdict                          |
|-------|------------|----------|----------------------------------|
| 1     | 1770.43    | 1561.32  | <i>best</i>                      |
| 2     | 1770.39    | 1561.29  | <i>best</i> (+0.003%)            |
| 3     | 1770.37    | 1561.27  | <i>best</i> (+0.001%)            |
| ...   |            |          |                                  |
| 10    | 1770.30    | 1561.21  | <i>best</i> (+0.007% cumulative) |

**Total val-loss improvement in 10 epochs: 0.007%.** This is essentially a model that did not learn anything from the 2.2M training examples.

### 2. Buffer sweep on fine-tuned Chronos (post-hoc tuning)

Even with generous buffer tuning, fine-tuned Chronos cannot compete:

| Buffer      | Fine-tuned cost (DKK) | MAE         |
|-------------|-----------------------|-------------|
| 1.00        | 14,078,178            | 2.06        |
| 1.40        | 11,365,149            | 2.28        |
| 1.80        | 9,740,286             | 2.71        |
| <b>2.00</b> | <b>9,194,415</b>      | <b>2.98</b> |

Best fine-tuned cost (9.19M) is still **40% worse than XGB cap=3 (6.56M)**.

### 3. Pre-trained vs fine-tuned comparison

To isolate whether fine-tuning itself helped, we also ran the pre-trained model zero-shot on the same test window:

| Buffer | Pre-trained cost (DKK) | Fine-tuned cost (DKK) | $\Delta$              |
|--------|------------------------|-----------------------|-----------------------|
| 1.00   | 14,075,900             | 14,078,178            | +0.02%                |
| 1.40   | 11,260,035             | 11,365,149            | <b>+0.93% (worse)</b> |
| 2.00   | 8,962,024              | 9,194,415             | <b>+2.59% (worse)</b> |

Fine-tuning systematically *worsened* performance. The 31 minutes of A100 compute moved the model in the wrong direction.

### 4. Chronos size sweep (does scaling help?)

We tested all four Chronos-Bolt sizes zero-shot:

| Size  | Params | Test MAE | Best cost (buf=3.0) |
|-------|--------|----------|---------------------|
| Tiny  | 9M     | 1.29     | 9,905,764           |
| Mini  | 21M    | 1.28     | 9,980,188           |
| Small | 48M    | 1.27     | 9,953,889           |
| Base  | 205M   | 1.28     | 9,945,868           |

**Going from 9M → 205M params changed nothing.** All sizes cluster at ~9.9M DKK best cost. Noise-level variance.

### 5. MLP (same 27 features as XGB)

We also trained a simple PyTorch MLP (3 layers, 512 hidden, SiLU, dropout 0.1) on the exact same features as XGB, to isolate “neural vs. tree” from “foundation vs. supervised”:

| Config                        | Test MAE    | Test cost (DKK)  |
|-------------------------------|-------------|------------------|
| MLP unconstrained             | <b>1.23</b> | 5,938,695        |
| MLP cap=3.0                   | —           | 7,615,921        |
| XGB unconstrained (reference) | 1.46        | <b>5,906,055</b> |
| XGB cap=3.0 (reference)       | —           | <b>6,558,810</b> |

**MLP achieves lower MAE but higher business cost** than XGB. MSE training is neutral on direction of error; asymmetric cost (waste cheap, stockout expensive) requires directional bias, which XGB gets from its tree splits. MLP needs bigger buffers to compensate, leaking more value on waste.

## 6. TimesFM zero-shot (failed)

Google’s TimesFM (200M/500M params, 100B time points pretrain) does not support Python 3.12 (Colab’s default). `timesfm==1.3.0` requires Python  $<3.12, \geq 3.10$ . Skipped.

## 7. Neural time-series architectures (darts: TFT / N-HiTS / DeepAR)

Tested on top-1500 series subset (high-volume), same train/val/test split. XGB re-evaluated on same subset for apples-to-apples.

| Model                                        | Test MAE    | Best cost (1500 subset) | Train time   |
|----------------------------------------------|-------------|-------------------------|--------------|
| <b>XGB per-group cap=3</b>                   | <b>7.02</b> | <b>2,200,459</b>        | <b>1.5 s</b> |
| DeepAR (2-layer LSTM, 64 hidden, 10 ep)      | 7.72        | 2,494,011               | 29.8 s       |
| N-HiTS (3 stacks, 2 layers, 256 wide, 10 ep) | 6.87        | 2,725,403               | 39.9 s       |
| TFT (Temporal Fusion Transformer, 10 ep)     | 8.84        | 6,279,909               | 178.7 s      |

**Observations:** - **XGB still wins** on the subset (2.20M vs 2.49M for DeepAR = 13% better). - **N-HiTS has the lowest MAE** (6.87) but loses on business cost — another case where MAE doesn’t predict cost under asymmetric loss. - **TFT underperformed** badly despite being the architecture most likely to compete (uses exogenous features). Default darts hyperparameters  $\times 10$  epochs is not enough; likely needs  $\sim$ day of hyperparameter search to compete. - **DeepAR is the strongest neural contender** — probabilistic output gives natural distributions for buffer selection, trains in 30 s. Worth deeper exploration if neural is required for a future product need (e.g., needing interpretable prediction intervals).

## Why Fine-Tuning Failed

### 1. No exogenous features

Chronos is a univariate forecaster. It sees only the raw demand series of each (store, item) pair — 64 days of integer counts. It cannot see: - Day-of-week - Shelf life - Storage type - Item price - Store chain/area - Rolling stats

XGB sees all of these as 27 explicit features. For any problem where side information is as predictive as the series itself, univariate foundation models lose.

### 2. Asymmetric cost function mismatch

Chronos is trained to predict the **distribution** of future values (quantile loss). When we compute business cost via  $\text{waste} + 1.5 \times \text{stockout}$ , we penalize under-predictions more than over-predictions. Tree models can learn to shift predictions upward; sequence-to-sequence transformers can’t easily without explicit reward shaping.

### 3. Integer count data outside pretraining distribution

Chronos was pretrained on  $\sim 1\text{T}$  time points of smooth continuous signals (finance, electricity, weather). Our data is mostly 0–10 integer counts with weekend/weekday shifts. The tokenizer allocates most of its budget to irrelevant-to-us ranges.

### 4. Loss aggregation bug (likely)

The epoch-level train/val loss reported ( $\sim 1770, \sim 1561$ ) barely moved across 10 epochs, while per-batch losses ( $\sim 1.2$ – $1.5$ ) moved normally. This strongly suggests the epoch aggregation sums per-batch loss over a fixed large reference

term (maybe outlier-clip penalty), so the relative movement in the aggregated value is invisible. The model may actually be learning on per-batch scale, but we can't use the epoch number to decide when to early-stop.

Even if this is just a display bug, **the downstream test cost got worse**, so the real learning signal was either zero or harmful.

## 5. Outlier clipping + short context

We clipped demand at the 99th percentile (73 units) to stabilize training. This removes exactly the information the model needs for the items that matter most (high-demand outliers). Short context (64 days = ~1.5 months) also gives the model very little seasonal structure to latch onto.

## What We Didn't Try (and Why)

| Idea                                            | Why skipped                                                                                                 |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Chronos-Bolt Base fine-tune (205M)              | Zero-shot Base ties Tiny/Mini/Small. Bigger won't fix pipeline issues. ~4hr on A100 for no expected upside. |
| TimesFM fine-tune                               | Package doesn't support Colab Python 3.12.                                                                  |
| Moirai / TimeGPT / Lag-Llama                    | Same univariate problem as Chronos; diminishing returns to try more FMs.                                    |
| Longer context (128, 256, 512 days)             | We only have ~155 days of train data; longer context offers no benefit.                                     |
| TFT (Temporal Fusion Transformer, multivariate) | Would take ~30 min. Likely hits ~XGB level but doesn't beat it. Good next experiment for future work.       |
| Chronos Small/Base fine-tune with loss-agg fix  | Worth 1-2 hours to investigate the aggregation bug; deferred to a future sprint.                            |
| TFT with serious hyperparameter search          | Would take ~1 day; even with covariates + 20 epochs TFT lost by 85%. Diminishing returns.                   |
| N-HiTS ensemble with XGB residual correction    | Combine N-HiTS base prediction + XGB residual learner. Could beat either individually. ~1 day experiment.   |

## 8. Darts models WITH covariates (proper setup)

Second pass on TFT/DeepAR/N-HiTS adding `future_covariates` (dow, is\_weekend, dom, month) and `static_covariates` (shelf\_life, storage\_type, perishability, chain\_id, area\_id, seasonal). 20 epochs.

| Model                      | MAE  | Best cost (1500 subset) | Train time   | vs XGB ref | $\Delta$ vs no-cov        |
|----------------------------|------|-------------------------|--------------|------------|---------------------------|
| <b>XGB per-group cap=3</b> | 7.02 | <b>2,200,459</b>        | <b>1.5 s</b> | —          | —                         |
| DeepAR + future covs       | 8.30 | 3,025,312               | 84 s         | +37.5%     | +21% (worse than no-cov)  |
| N-HiTS (bigger, longer)    | 6.99 | 3,135,556               | 169 s        | +42.5%     | +15% (worse than no-cov)  |
| TFT + static + future covs | 8.74 | 4,059,438               | 821 s        | +84.5%     | <b>-35% (much better)</b> |

**Key findings:** - TFT got the expected boost from covariates (-35% cost), confirming the architecture wants side information. But it remains 85% worse than XGB. - DeepAR and N-HiTS got worse with more epochs — overfit to train. - 14 minutes of TFT training + covariates + 20 epochs still produces a model that XGB beats in 1.5 seconds.

This is the final nail: **no configuration of any neural architecture we tested beats XGBoost on this problem.** Tree-based gradient boosting is the right tool for retail demand forecasting with rich side features.

---

## Cost Summary (GPU time spent)

| Experiment                                | GPU time       | Outcome                        |
|-------------------------------------------|----------------|--------------------------------|
| Chronos fine-tune 10 epochs               | 31 min         | Worsened by 2.6%               |
| Chronos-Bolt size sweep (4 sizes)         | 1.5 min        | All identical, all lose to XGB |
| MLP training                              | 6.5 s          | Ties XGB unconstrained         |
| XGB-GPU baseline                          | 3.1 s          | Winner                         |
| <b>Total A100 time on fine-tune track</b> | <b>~35 min</b> | <b>No improvement</b>          |

---

## Conclusion & Recommendations

1. **Do not deploy Chronos for this problem.** Zero-shot is 37% worse than XGB; fine-tuning makes it worse still.
  2. **Foundation models for time series are not automatically better.** They win on problems where the target series contains all the signal. Retail demand with rich side features (shelf life, store type, day-of-week) is not one of those problems.
  3. **Invest in XGB features, not new models.** A 1% improvement in XGB feature engineering (say, adding lagged-promo signals or local weather) is worth more than the entire Chronos fine-tune endeavor.
  4. **If revisiting neural:** try TFT or a small attention model that consumes the 27-feature vector, with cost-weighted training (asymmetric loss). Budget: 1 day. Expected: tie XGB, not beat.
  5. **The one thing fine-tuning is useful for** in this codebase: domain adaptation for forecasting in a *new market* (different currency, cuisine, seasonality patterns) where we don't have enough data to train a fresh XGB. Not the case today.
- 

## Artifacts

- notebooks/improvement\_3\_chronos\_finetune.py — fine-tune pipeline
- notebooks/fast\_chronos\_sizes.py — size sweep
- notebooks/fast\_mlp\_vs\_xgb.py — MLP baseline
- results\_chronos\_finetune/ — fine-tune logs + saved model (chronos-bolt-mini-finetuned-v2/)
- results\_chronos\_sizes/chronos\_sizes\_results.json
- results\_mlp/mlp\_results.json
- Full session log: data/colab\_results/script3.log